Final Mastery Report

Overview

This project implements the `v1-album-store` ChaosArena assignment in Go and deploys it publicly on AWS. The final system satisfies the full assignment contract and reached a final ChaosArena score of `190/190`.

Final public base URL:

- `http://final-mastery-album-store-1120023327.us-west-2.elb.amazonaws.com`

Final successful ChaosArena run:

- Nickname: `kaddy`
- Score: `190/190`

Assignment Goals

The assignment required a live REST API with:

- `GET /health`
- `PUT /albums/:album_id`
- `GET /albums/:album_id`
- `GET /albums`
- `POST /albums/:album_id/photos`
- `GET /albums/:album_id/photos/:photo_id`
- `DELETE /albums/:album_id/photos/:photo_id`

Key behavioral requirements included:

- exact health response body
- idempotent album upsert
- async photo upload
- synchronous per-album `seq` assignment
- real fetchable completed photo URL
- delete behavior that removes metadata and file access
- a live deployment accessible via one public base URL

Initial Architecture

The project started as a Go service with:

- local SQLite for metadata
- local filesystem storage for uploaded and processed photos
- a background worker pool for async photo processing
- Docker support for deployment

This version was good for correctness but weak for load testing because:

- ECS task-local disk is not durable
- SQLite serializes writes under load
- one task and local storage create throughput limits
- photo processing involved unnecessary file-copy overhead

Major Bottlenecks We Faced

1. Empty Album List Returned `null`

Problem:

- `GET /albums` returned `null` instead of `[]` when empty

Impact:

- this would fail correctness scenario `S5`

Fix:

- initialized the list with `make([]Album, 0)` instead of leaving it nil

2. Delete vs Worker Race

Problem:

- deleting a photo while the worker was still processing it could leave orphaned files

Impact:

- risk in advanced delete scenarios `S7` to `S9`

Fixes:

- changed delete behavior to use atomic row deletion patterns
- staged output carefully and cleaned up temp artifacts if rows disappeared
- added delete-during-processing regression coverage

3. Album Upsert Race

Problem:

- concurrent album creates could both behave like creators and return the wrong status code

Impact:

- correctness risk under concurrent create scenarios

Fix:

- made album upsert atomic so the first write returns `201` and later ones return `200`

4. Dockerfile Build Risk

Problem:

- `go.sum` was not copied before `go mod download`

Impact:

- Docker builds could fail, which would make deployment impossible

Fix:

- updated the Dockerfile to copy both `go.mod` and `go.sum` before downloading dependencies

5. TLS Failure in ECS Runtime

Problem:

- the ECS container initially failed AWS SDK calls because the runtime image lacked CA certificates

Impact:

- the service started but AWS requests failed with certificate errors

Fix:

- installed `ca-certificates` in the runtime image

6. DynamoDB Album Upsert Error

Problem:

- DynamoDB update expressions used raw attribute names that could trigger validation errors

Impact:

- album creation returned internal errors in AWS mode

Fix:

- switched to expression attribute names like `#title`, `#description`, and `#next_seq`

7. S3 Upload Failure Due to Missing Content Length

Problem:

- S3 `PutObject` rejected streamed request bodies without `Content-Length`

Impact:

- photo upload returned `500`

Fix:

- wrote incoming photo content to a temp file
- reopened the file and uploaded it with a known size

8. Load-Test Throughput Bottleneck in POST Handler

Problem:

- photo uploads were still sending data to S3 before returning `202`

Impact:

- `S12`, `S14`, and `S15` lost many points because POST accept and completion times were too high

Fix:

- changed the request path so it:
  - receives the multipart body
  - stores it temporarily on local disk
  - reserves the per-album sequence in DynamoDB
  - writes metadata to DynamoDB
  - returns `202` immediately
- moved the S3 upload to the background worker

9. Remaining Photo Completion Bottleneck

Problem:

- after the request-path optimization, large and concurrent uploads still needed faster background transfer performance

Impact:

- remaining load score gap after the first big improvement

Fix:

- switched worker-side S3 transfer to the AWS S3 transfer manager
- increased ECS resources and service concurrency

Architecture Evolution

Phase 1: Local Correctness-Focused Service

- Go HTTP server
- SQLite metadata
- local photo storage
- async workers

Result:

- strong correctness foundation
- poor long-term load characteristics

Phase 2: Cloud-Native Storage

We migrated to:

- DynamoDB for album and photo metadata

- S3 for photo storage
- ECS Fargate behind an ALB
- Terraform-managed AWS deployment in `us-west-2`

This solved:

- non-durable task-local file storage
- SQLite write contention
- single-node metadata bottlenecks

Phase 3: Throughput Tuning

We improved performance by:

- reducing request-path work for uploads
- keeping `202 Accepted` fast
- scaling ECS task count upward
- increasing task CPU and memory
- increasing background worker capacity
- using the S3 transfer manager for concurrent object upload in workers

AWS Infrastructure

Final deployed AWS components:

- ECS Fargate service
- Application Load Balancer
- ECR repository
- S3 bucket for photo storage
- DynamoDB table for albums
- DynamoDB table for photos
- CloudWatch log group
- autoscaling configuration

Terraform files:

- [main.tf](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\terraform\main.tf)
- [variables.tf](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\terraform\variables.tf)
- [outputs.tf](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\terraform\outputs.tf)

Score Progression

First Strong Correctness Run

Run:

- `20260413T000023Z_e10d54e4-e662-4fd1-90f6-b65375955adc`

Score:

- `123/190`

Breakdown:

- correctness: `110/110`
- load: `13/80`

Important metrics:

- `S12`: `0/15`, `p95_ms: 17672`
- `S14`: `2/15`
- `S15`: `2/20`

Interpretation:

- correctness was solved
- load bottlenecks were concentrated in photo-heavy scenarios

After Moving S3 Upload Out of the Request Path

Run:

- `20260413T001549Z_ffa387f3-5787-4a2e-8845-f9171b7236f7`

Score:

- `177/190`

Breakdown:

- correctness: `110/110`
- load: `67/80`

Important improvements:

- `S11`: `12/15`
- `S12`: `8/15`
- `S13`: `15/15`
- `S14`: `13/15`
- `S15`: `19/20`

Interpretation:

- the request-path optimization produced the biggest score jump
- most of the remaining gap was in concurrent photo completion under load

Final Optimized Run

Run:

- `20260413T002849Z_196501ac-7ab8-4576-958d-c6599de39884`

Score:

- `190/190`

Final scenario results:

- `S11`: `15/15`, `p95_ms: 12`
- `S12`: `15/15`, `p95_ms: 1412`
- `S13`: `15/15`, `p95_ms: 16`
- `S14`: `15/15`
- `S15`: `20/20`

Interpretation:

- correctness remained perfect
- the final load bottlenecks were removed through worker-side transfer optimization and larger ECS capacity

Exact Final Improvements That Raised the Score to 190

The final jump from `177/190` to `190/190` came from a targeted last optimization pass focused on the remaining load-test bottlenecks.

1. Faster Worker-Side S3 Uploads

In [main.go](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\main.go), the AWS backend was updated to initialize a shared S3 transfer manager uploader.

This mattered because:

- earlier versions had already moved S3 upload work out of the request path
- however, background workers were still using a simpler single S3 upload call
- under concurrent photo load, that left performance on the table for `POST -> completed` latency

In [aws_backend.go](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\aws_backend.go), the worker switched from plain `s3.PutObject` to the transfer manager uploader.

Impact:

- better throughput during concurrent photo processing
- lower completion latency in `S12`
- better upload-side performance in `S14`
- faster large-object completion in `S15`

2. Larger ECS Task Size

In [terraform/variables.tf](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\terraform\variables.tf), ECS task resources were increased:

- CPU from `1024` to `2048`
- memory from `2048 MiB` to `4096 MiB`

Impact:

- each ECS task could process more uploads in parallel
- worker contention dropped under heavy concurrent photo traffic
- photo-completion throughput improved during the load scenarios

3. More ECS Service Capacity

In [terraform/variables.tf](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\terraform\variables.tf), service scaling was increased:

- desired task count from `4` to `6`
- max autoscaling capacity from `8` to `12`

Impact:

- more replicas were available before the load tests started
- the service handled concurrent metadata and photo traffic more evenly
- this especially helped mixed scenarios where uploads and metadata operations ran together

4. More Background Worker Concurrency

In [terraform/variables.tf](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\terraform\variables.tf), background workers were increased:

- `MAX_WORKERS` from `16` to `32`

Impact:

- more queued photo jobs could be processed at once
- photo backlog cleared faster between and during upload-heavy scenarios
- completion p95 improved enough to close the last scoring gap

5. Why These Changes Improved the Final Score

Before the last optimization pass, the service already had:

- full correctness
- fast `202 Accepted` behavior
- strong metadata performance

The remaining gap was mostly in photo-heavy completion performance. The final pass improved that exact area:

- `S12` reached full points because concurrent photo completion became fast enough
- `S14` reached full points because metadata traffic and photo traffic both stayed responsive under concurrency
- `S15` reached full points because both accept latency and completion latency for large uploads were low enough

The final load results reflected those changes directly:

- `S11`: `15/15`

- `S12`: `15/15`
- `S13`: `15/15`
- `S14`: `15/15`
- `S15`: `20/20`

How We Used ChaosArena Debugging

ChaosArena's `/runs/<run_id>` response acted as the debugging API for this project.

We used it to inspect:

- per-scenario correctness results
- p95 and p99 metrics
- split load metrics in `S14` and `S15`
- progression between submissions

Examples of how the debug data guided fixes:

- very high `S12` and `S15` latencies showed that POST and completion paths were too expensive
- strong correctness with poor load indicated that the API logic was correct but the architecture needed tuning
- improvement in `accept_p95_ms` after moving S3 upload to the worker confirmed that the request path was the main bottleneck
- remaining `S12` and upload-completion cost in `S14` indicated that background transfer speed and service scale still needed work

Key Lessons Learned

1. Correctness and load performance require different architecture choices.
2. A design that passes all contract scenarios may still score poorly if the request path does too much work.
3. Returning `202` quickly is not enough by itself; POST-to-completed latency under concurrency matters.
4. Moving heavy cloud I/O out of the request path produced the largest performance gain.
5. Scaling infrastructure only becomes effective once the application path can use that capacity efficiently.
6. Iterating with real benchmark feedback is much faster than guessing.

Final State

The final service:

- passes all correctness scenarios
- achieves full load score
- is deployed publicly on AWS in `us-west-2`
- uses S3 and DynamoDB instead of local persistent state
- scales across multiple ECS tasks

Core files:

- [main.go](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\main.go)

- [aws_backend.go](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\aws_backend.go)
- [main_test.go](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\main_test.go)
- [Dockerfile](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\Dockerfile)
- [README.md](C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\final-mastery\README.md)

Conclusion

We started with a correctness-oriented Go service and evolved it into a high-performance cloud-backed deployment. The biggest breakthrough came from separating upload acceptance from cloud storage transfer, then pairing that with stronger ECS scaling and faster worker-side S3 uploads. That progression moved the project from `123/190` to `177/190` and finally to a full `190/190`.